

# Claude Code: programación agentic frente a Chat y Cowork

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

**ES** Sesión 3 de 12 · Notas en español (es-ES)

Tras el mapa de certs, Vijay pasó al producto por el que existe este curso: **Claude Code**, la herramienta de programación con IA de Anthropic. Empezó como asistente y ahora es un cliente **agentic**: lee un proyecto entero (RAG sobre el espacio de trabajo), edita muchos archivos, ejecuta comandos de terminal, pruebas, git/PR y habla con Jira, Figma y otras herramientas. No es «Chat con una carpeta». No es el ordenador de facturas de Cowork. Es el agente de SDLC.

## Asistente frente a agente (el contraste que trazó)

| Asistente de programación antiguo                       | Claude Code como agente                                            |
|---------------------------------------------------------|--------------------------------------------------------------------|
| Sugiere en el archivo actual; espera la siguiente tecla | Entiende el repo; lee/modifica varios archivos                     |
| Contexto limitado, fácil perder el hilo                 | Recupera trozos relevantes; puede planificar y luego ejecutar      |
| Recomienda; vosotros siempre pegáis                     | Actúa con vuestro permiso: shell, pruebas, PR                      |
| La identidad temprana de GitHub Copilot en su relato    | Copilot también ha ido hacia agentes — sigue sin ser este producto |

Un solo prompt puede apuntar ahora a backend, frontend, scripts de base de datos, Docker, Terraform, Jenkins — un espacio desplegable. Esa es la promesa de productividad: menos faena repetida, sprints más rápidos, formato más limpio si codificáis estándares. Los repos brownfield existentes igual necesitan buenos prompts (pregunta de Lalit); el boilerplate greenfield es la demo fácil.

## Cómo corre el bucle

1. Prompt de usuario — la intención puede ser «solo el plan» o «construye el proyecto».
2. El modelo seleccionado (Sonnet / Opus / Haiku / otros que habilite la org) interpreta la intención.
3. Explorar el proyecto para contexto.
4. Crear un plan de ejecución.
5. Ejecutar con herramientas integradas: leer/editar archivos, shell, MCP/conectores de terceros, git.
6. Verificar (p. ej. leer logs del terminal como herramientas de E/S) y terminar o preguntaros.

Los logs del terminal son una **herramienta**: el agente lee éxito/fallo y continúa. Es la misma separación planificar/ejecutar que Cowork, con el sistema de archivos y el compilador como manos.

## Vías de instalación que demostró

Prerrequisitos: inicio de sesión en la consola Anthropic con Pro/Max/Team/Enterprise; **Node.js 18+**; un terminal. Opcional: VS Code (lo prefirió al editor de Claude Desktop porque las extensiones hacen real el entorno). Dos instalaciones upstream: script `curl` del fabricante o `npm global`. Tras instalar, el comando CLI

es Claude en la carpeta que debe ser el espacio de trabajo. Inicio de sesión/autorización por la consola. Notas de seguridad al arrancar: los modelos se equivocan; riesgo de inyección de prompts — usadlo solo en código en el que confiéis. Lo beta no va en el instalador por defecto; UST querrá instalación por portal/registro privado, no un curl aleatorio en un portátil de producción. Windows: en el chat había que poner Claude en el PATH / variables de entorno.

## CLI y luego VS Code

Asistente de primer arranque: tema del terminal, confiar en esta carpeta git, mostrar versión / usuario / modelo por defecto. Ya aparecen comandos slash: `/init` para CLAUDE.md, `/release-notes`, `/model`, `/exit`. Luego instaló la extensión de VS Code de **Anthropic** (no un editor clon), autorizó Claude.ai y usó el icono de Code para sesiones. El botón más = sesión nueva; adjuntar archivos abiertos, PDF, búsqueda web; algunas funciones con glifo de terminal os siguen bajando al terminal integrado porque la UI no ha llegado.

### UI FRENTE A DOCS

Los IDs de extensión, los modelos por defecto (en clase tenía Sonnet 5) y qué comandos slash viven solo en el TTY cambiarán. Preferid **la documentación de Claude Code** la semana del examen.

## Cuándo elegir Code en lugar de Chat o Cowork

- **Chat:** explicar un algoritmo, redactar un correo, sin repo.
- **Cowork:** reordenar archivos de oficina locales; trabajos de varios pasos no de desarrollo.
- **Code:** el espacio es control de código; necesitáis pruebas, PR, automatizar código limpio o generar la propia app.

## Ángulo de examen

Si el enunciado es un desarrollador que debe implementar, depurar o cablear CI, el producto nombrado es Claude Code (o la API), no Cowork. Si el enunciado es «organizar PDF en disco para un BA», no presumáis de Code. El encaje de producto es una competencia Foundation.

## Documentación oficial

### • Resumen de Claude Code

<https://code.claude.com/docs/en/overview>

Instalación, CLI, IDE — la fuente de verdad.

### • Modelos

<https://docs.claude.com/en/docs/about-claude/models>

A qué puede apuntar `/model` en vuestro tenant.

### • Resumen de uso de herramientas

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Las herramientas de shell y archivos son esta idea dentro de Code.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Jira, Figma y servidores a medida desde el agente.

- **Resumen de la API**

<https://docs.claude.com/en/api/overview>

Cuando la respuesta es «no uses el agente GUI; llama a Messages».

---

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.